

Contents

Chapter 1

Installation and setup

1.1 Installing C++ Compiler for windows

- Go to: <http://mingw-w64.org/doku.php/download/mingw-builds>
- Go to: Downloads, find the build, download and run executable.
- Set the environment variable:
 - Control panel → Edit system environment variables.
 - Environment variables → System → Path → Edit.
 - New → Browse → < go to your instalation dir > → OK.
- Go to CMD: type `c++ --version` → Should print version.

1.2 VSCode Project Setting Up

Inside the `.vscode` directory add:

- `c_cpp_properties.json`:

```
1 {
2     "configurations": [
3         {
4             "name": "Win32",
5             "includePath": [
6                 "${workspaceFolder}/**"
7             ],
8             "defines": [
9                 "_DEBUG",
10                "UNICODE",
11                "_UNICODE"
12            ],
13            "browse": {
14                "path": [
15                    "${workspaceRoot}",
16                    "C:\\Program Files\\mingw-w64\\mingw64\\bin\\gcc.exe"
17                ]
18            },
19            "compilerPath": "C:\\Program Files\\mingw-w64\\mingw64\\bin\\gcc.exe",
20            "cStandard": "gnu18",
```

```

21         "cppStandard": "gnu++14"
22     }
23 ],
24     "version": 4
25 }

```

- launch.json:

```

1 {
2     "version": "0.2.0",
3     "configurations": [
4         {
5             "name": "g++.exe - Compilar y depurar el archivo activo",
6             "type": "cppdbg",
7             "request": "launch",
8             "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
9             "args": [],
10            "stopAtEntry": false,
11            "cwd": "${workspaceFolder}",
12            "environment": [],
13            "externalConsole": false,
14            "MIMode": "gdb",
15            "miDebuggerPath": "C:\\Program
↳ Files\\mingw-w64\\mingw64\\bin\\gdb.exe",
16            "setupCommands": [
17                {
18                    "description": "Habilitar la impresin con sangra para gdb",
19                    "text": "-enable-pretty-printing",
20                    "ignoreFailures": true
21                }
22            ],
23            "preLaunchTask": "C/C++: g++.exe build active file"
24        }
25    ]
26 }

```

- In order to establish the formatting style put the following in settings.json:

```

1 {
2     "C_Cpp.clang_format_fallbackStyle": "{ BasedOnStyle: LLVM, UseTab: Never,
↳ IndentWidth: 4, TabWidth: 4, BreakBeforeBraces: Attach,
↳ AllowShortIfStatementsOnASingleLine: false, IndentCaseLabels: false,
↳ ColumnLimit: 0, AccessModifierOffset: -4 }",
3     "emmet.showSuggestionsAsSnippets": true,
4     "files.associations": {
5         "*.rmd": "markdown",
6         "iostream": "cpp"
7     }
8 }

```

- task.json:

```

1 {
2     "version": "2.0.0",

```

```
3  "tasks": [  
4      {  
5          "label": "C/C++: g++.exe build active file",  
6          "type": "shell",  
7          "command": "C:\\Program Files\\mingw-w64\\mingw64\\bin\\g++.exe",  
8          "args": [  
9              "-g", "main.cpp", "-o", "a.exe" // , "&&", "main"  
10         ],  
11         "problemMatcher": [  
12             "$gcc"  
13         ],  
14         "presentation": {  
15             "echo": false,  
16             "reveal": "always",  
17             "focus": false,  
18             "panel": "shared",  
19             "showReuseMessage": true,  
20             "clear": false  
21         },  
22         "group": {  
23             "kind": "build",  
24             "isDefault": true  
25         }  
26     }  
27 ]  
28 }
```

Chapter 2

Operator overloading

2.1 Section overview

- Operator overloading allows the user to use the operator with objects.
- Overloading operators as member functions.
- Overloading operators as global functions.
- Overloading stream insertion (<<) and extraction operators (>>).

2.2 What is operator overloading?

- Using traditional operators such as +, =, *, etc. with user-defined types.
- Allows user defined types to behave similar to built-in-types.
- Can make code more readable and writeable.
- Not done automatically (except for the assignment operator). Operator overloading operations must be explicitly defined.

2.2.1 Example

- Suppose we have a Number class that models any number, so we can do anything with numbers such as complex numbers, integers, decimals, etc.
- For this kind of thing we would need to make this class call functions or methods to multiply - for example - and this does not feel natural.

```
1 Number result = multiply(add(a, b), divide(c, d));  
2 Number result = (a.add(b)).multiply(c.divide(d));
```

- For C++ it does not make sense to do operations with two objects, thus you need to tell the C++ compiler what you mean when you are doing an operation with two objects, you can do this with methods - like exemplified above - or with overloading operators.
- Only overload operators when it makes sense, otherwise the code is not going to be readable and calling a function/method is better.
- Lets overload the +, * operators.

```
1 Number result = (a + b) * (c /d);
```

- Overloading operators is just for easier programming, behind the scenes we are still calling functions and methods.

2.2.2 What operators can be overloaded?

- The majority of C++'s operators can be overloaded.
- The following operators cannot be overloaded:

::	Scope resolution operator
?:	Conditional operator
.*	member operator
.	dot operator
sizeof	sizeof operator

2.2.3 Some basic rules

- Precedence and associativity cannot be changed. You cannot make operators so that one operation takes place before another when the operators do not behave like that normally.
- 'arity' cannot be changed (i.e. can't make the division operator unary), this means that if the operator expected to do something with two arguments then when you overload it, it expects two arguments. Example: $a + b$ expects argument a and argument b, you cannot overload it so that it takes in a third or takes in just one, it remains at 2.
- Can't overload operators for primitive type (`int`, `double`, etc).
- Can't create new operators.
- `[]`, `()`, `->`, and the assignment operator (`=`) must be declared as member methods.
- Other operators can be declared as member methods or global functions.

2.2.4 Examples

<pre>1 int a = b + c; 2 a < b; 3 std::cout << a;</pre>	<pre>1 std::string s1 = s2 + s3; 2 s1 < s2; 3 std::cout << s1;</pre>
<pre>1 double a = b + c; 2 a < b; 3 std::cout << a;</pre>	<pre>1 std::string my_string = s2 + ↪ s3; 2 s1 < s2; 3 s1 == s2; 4 std::cout << s1;</pre>

2.2.5 The class we will be using through out this section

Class definition in Mystring.h:

```
1 #ifndef MYSTRING_H
2     #define MYSTRING_H
3
4 class Mystring {
5 private:
6     char *str;
7 public:
8     Mystring();
9     Mystring(const char *s);
10    Mystring(const Mystring &source);
11    ~Mystring();
12
13    void display() const;
14    int get_length() const;
15    const char *get_str() const;
16 };
17 #endif
```

Class implementation in Mystring.cpp:

```
1 // no args constructor.
2 Mystring::Mystring() : str(nullptr) {
3     // Make an empty string
4     str = new char[1];
5     *str = '\0';
6 }
7
8 // copy constructor.
9 Mystring::Mystring(const char *s) : str{nullptr} {
10     if (s == nullptr) {
11         str = new char[1];
12         *str = '\0';
13     }
14     else {
15         str = new char[strlen(s) + 1];
16         strcpy(str, s);
17     }
18 }
19
20 // destructor.
21 Mystring::~Mystring() {
22     delete[] str;
23 }
24
25 // print the string.
26 void Mystring::display() const {
27     std::cout << str << " : " << get_length() << std::endl;
28 }
29
30 // string length.
31 int Mystring::get_length() const { return strlen(str); }
```



```

32
33 // string getter.
34 const char *Mystring::get_str() const { return str; }

```

2.3 Overloading the assignment operator

- We will overload the assignment operator to mean deep copy from a string object into another string object.
- C++ provides a default assignment operator used for assigning one object to another.

```

1 Mystring s1 {"frank"};
2 Mystring s2 = s1; // NOT assignment it is the same as Mystring s2 {s1}; this is
  ↳ done my constructors to initialize.
3 s2 = s1; // assignment, once initialized the assignment can be called.

```

- Default is memberwise assignment (shallow copy):
 - If we have raw pointer data member we must use deep copy.

2.3.1 Overloading the copy assignment operator (deep copy)

- The assignment must be overloaded as a member function. So we provide a member prototype:

```
Type &Type::operator=(const Type &rhs);
```

- Replace “Type” with the name of the class in question.

```

1 Mystring &Mystring::operator=(const Mystring &rhs);
2 s2 = s1; // we write this.
3 s2.operator=(s1); // operator= method is called.

```

- Behind the scenes the method “.operator=” is called.

Overloading the copy assignment operator (deep copy) step by step.

```

1 Mystring &Mystring::operator=(const Mystring &rhs) {
2     if (this == &rhs) {
3         return *this;
4     }
5     delete[] str;
6     str = new char[strlen(rhs.str) + 1];
7     strcpy(str, rhs.str);
8     return *this;
9 }

```

- First, check for self assignment, if the right hand side (rhs) is exactly the same as the left hand side (lhs) then we terminate the function by returning `*this` since we already made the copy and there is nothing else to do. We check the address of both pointers, if they are the same, then we assigning an object to itself.

```

1 if (this == &rhs) { // p1 = p1;
2     return *this; // return current object.
3 }

```

- Deallocate storage for `this->str` since we are overwriting it. Since we need to deallocate this data because we will allocate the string somewhere else. This is to avoid memory leaks.

```

1 delete[] str;

```

- Allocate storage for the deep copy.

```

1 str = new char[std::strlen(rhs.str) + 1];

```

- Perform the copy. Use `strcpy()` function to copy the string one character at a time.

```

1 std::strcpy(str, rhs.str);

```

- Return the current by reference to allow chain assignment. Remember chain assignment is when we have several assignments in the same line (example: `s1 = s2 = s3 = s4;`)

```

1 return *this; // s1 = s2 = s3 = s4;

```

2.3.2 Example

Now lets add the overloaded assignment operator to the `Mystring` class. This is in `Mystring.h`.

```

1 #ifndef MYSTRING_H
2     #define MYSTRING_H
3 class Mystring {
4 private:
5     char *str;
6 public:
7     Mystring();
8     Mystring(const char *s);
9     Mystring(const Mystring &source);
10    ~Mystring();
11
12    // copy assignment (overloaded).
13    Mystring &operator=(const Mystring &rhs);
14
15    void display() const;
16    int get_length() const;
17    const char *get_str() const;
18 };
19 #endif

```

On `Mystring.cpp` I add this implementation.

```

1 Mystring &Mystring::operator=(const Mystring &rhs) {
2     std::cout << "Copy assignment" << std::endl;
3     if (this == &rhs) {
4         return *this;
5     }

```

```

6   delete[] this->str;
7   str = new char[strlen(rhs) + 1];
8   strcpy(this->str, rhs.str);
9   return *this;
10 }

```

On the main.cpp:

```

1  #include <iostream>
2  #include <cstring>
3  #include "Mystring.h"
4  int main() {
5      Mystring a {"hello"};
6      Mystring b;
7      b = a;
8      b = "This is a test.";
9      return 0;
10 }
11 /* OUTPUT:
12 Copy assignment
13 Copy assignment
14 */

```

2.4 Overloading the move assignment operator

- Move assignment operator is also denoted with the '='.
- You can choose to overload the move assignment operator.
 - C++ will use the copy assignment operator if necessary.

```

1  Mystring s1;
2  s1 = Mystring {"Frank"}; // move assignment, this is an unnamed object that is
   ↳ constructed and has no name (this is an r value), by moving it to s1 it is
   ↳ granted a name.

```

- The assignment deep copy operator in the last section works with left value references, so values. The move assignment works with right value references. This means temporary unnamed objects that need assignment (move).
- You don't need to provide a move assignment operator, if you don't C++ will use the copy constructor by default. It is always recommended to provide a move assignment operator and a move constructor.
- The main difference between the move assignment operator and the copy assignment operator (seen in the last section) is that in one we are deep copying every character - in this case - of the string, while in this one we are just stealing the pointer from the right hand side and assigning it to the left.

2.4.1 Overloading the move assignment operator

- `Type &Type::operator=(Type &&rhs);`

```

1  // in the Mystring class.
2  Mystring &Mystring::operator=(Mystring &&rhs);

```

```

3 s1 = Mystring{"Joe"}; // move operator= called.
4 s1 = "Frank"; // move operator= called.

```

2.4.2 Overloading the move assignment operator - steps

```

1 Mystring &Mystring::operator=(Mystring &&rhs) {
2     if (this == &rhs) { // self assignment.
3         return *this;    // return current object.
4     }
5     delete[] str; // deallocate current storage.
6     str = rhs.str; // steal the pointer
7     rhs.str = nullptr; // null out the rhs object.
8     return *this;
9 }

```

- Check for self assignment:

```

1 if (this == &rhs) {
2     return *this;
3 }

```

- Deallocate storage for `this->str` since we are overwriting it and we don't want a memory leak.

```

1 delete[] str;

```

- Steal the pointer from the rhs object and assignment it to `this->str`:

```

1 str = rhs.str;

```

- Null out the rhs pointer:

```

1 rhs.str = nullptr;

```

- Return the current object by reference to allow chain assignment.

```

1 return *this;

```

2.4.3 Example

Comparing the move assignment operator and the copy assignment operator. On the `Mystring.h`:

```

1 #include <iostream>
2 #include <vector>
3 #include <string.h>
4
5 #ifndef MYSTRING_H
6     #define MYSTRING_H
7
8     class Mystring {
9     private:
10         char *str;
11     public:
12         Mystring(); // no args constructor.

```

```

13     Mystring(const char *s); // overloaded constructor.
14     Mystring(const Mystring &source); // copy constructor
15     ~Mystring(); // destructor
16     Mystring(Mystring &&source); // move constructor
17
18     Mystring &operator=(const Mystring &rhs); // copy assignment.
19     Mystring &operator=(Mystring &&rhs); // Move assignment.
20
21     void display() const;
22     int get_length() const;
23     const char *get_str() const;
24 };
25
26 // no args constructor.
27 Mystring::Mystring() : str(nullptr) {
28     // Make an empty string
29     str = new char[1];
30     *str = '\0';
31 }
32
33 // copy constructor.
34 Mystring::Mystring(const char *s) : str{nullptr} {
35     if (s == nullptr) {
36         str = new char[1];
37         *str = '\0';
38     }
39     else {
40         str = new char[strlen(s) + 1];
41         strcpy(str, s);
42     }
43 }
44
45 // destructor.
46 Mystring::~Mystring() {
47     delete[] str;
48 }
49
50 // move constructor
51 Mystring::Mystring(Mystring &&source) : str{source.str} {
52     source.str = nullptr;
53     std::cout << "Move constructor used" << std::endl;
54 }
55
56 // move assignment.
57 Mystring &Mystring::operator=(Mystring &&rhs) {
58     std::cout << "Using move assignment" << std::endl;
59     if (this == &rhs) {
60         return *this;
61     }
62     delete[] str;
63     str = rhs.str;
64     rhs.str = nullptr;
65     return *this;
66 }

```

```

67
68
69 // print the string.
70 void Mystring::display() const {
71     std::cout << str << ": " << get_length() << std::endl;
72 }
73
74 // string length.
75 int Mystring::get_length() const { return strlen(str); }
76
77 // string getter.
78 const char *Mystring::get_str() const { return str; }
79
80 // copy assignment operator overloading.
81 Mystring &Mystring::operator=(const Mystring &rhs) {
82     if (this == &rhs) {
83         return *this;
84     }
85     delete[] str;
86     str = new char[strlen(rhs.str) + 1];
87     strcpy(str, rhs.str);
88     return *this;
89 }
90 #endif

```

On the main.cpp:

```

1 #include <iostream>
2 #include <vector>
3 #include "Mystring.h"
4
5 using namespace std;
6
7 int main() {
8     Mystring a{"Hello"}; // overloaded constructor
9     a.display();
10    a = Mystring{"Hola"}; // overloaded constructor
11    a.display();
12    a = "Bonjour";
13    a.display();
14    return 0;
15 }
16
17 /* OUTPUT:
18 Hello: 5
19 Using move assignment
20 Hola: 4
21 Using move assignment
22 Bonjour: 7
23 */

```

2.5 Overloading operators as member functions

- Don't be confused by 'member functions' this just means methods.
- You can overload operators using methods or functions in a program, if you use method the operator will be overloaded for that object. In this case this is member function overloading

2.5.1 Unary operands

- Unary operators as member methods. (**++**, **--**, **-**, **!**)

```
ReturnType Type::operatorOp();
```

```
1 Number Number::operator-() const;
2 Number Number::operator++(); // pre-increment
3 Number Number::operator++(int); // post-increment
4 Number Number::operator!() const;
5
6 Number n1 {100};
7 Number n2 = -n1; // n1.operator-()
8 n2 = ++n1; // n1.operator++()
9 n2 = n1++; // n1.operator++(int)
```

- Example with the Mystring class.
 - Lets use the **operator-** to make lower case a upper case string.

```
1 Mystring larry1 {"LARRY"};
2 Mystring larry2;
3 larry1.display(); // LARRY
4 larry2 = -larry1; // larry1.operator-()
5 larry1.display(); // LARRY
6 larry2.display(); // larry
```

- Code to make a string minus mean lower case:

```
1 Mystring Mystring::operator-() const {
2     char *buff = new char[strlen(str) + 1];
3     strcpy(buff, str);
4     for (size_t i = 0; i < strlen(buff); i++) {
5         buff[i] = tolower(buff[i]);
6     }
7     Mystring temp {buff};
8     delete[] buff;
9     return temp;
10 }
```

2.5.2 Binary operators as member methods

- Binary operators are **+**, **-**, **==**, **!=**, **<**, **>**

```
ReturnType Type::operatorOp(const Type &rhs);
```

```

1 Number Number::operator+(const Number &rhs) const;
2 Number Number::operator-(const Number &rhs) const;
3 bool Number::operator==(const Number &rhs) const;
4 bool Number::operator<(const Number &rhs) const;
5
6 Number n1 {100}, n2 {200};
7 Number n2 = n1 + n2; // n1.operator+(n2)
8 n3 = n1 - n2; // n1.operator-(n2)
9 if (n1 == n2) ... // n1.operator==(n2)

```

- Example of implementation of equality operator in the Mystring class.

```

1 bool Mystring::operator==(const Mystring &rhs) const {
2     if (std::strcmp(str, rhs.str) == 0) {
3         return true;
4     }
5     else {
6         return false;
7     }
8 }

```

- Example of string concatenation using the + operator overloading:

```

1 Mystring larry {"Larry"};
2 Mystring moe {"Moe"};
3 Mystring result = larry + stooges; // larry.operator+(stooges);
4 result = moe + "is also a stooge"; // compiler error, moe.operator+("is also a
↳ stooge"); the right side needs to be the same type as the class you are using.
5 result = "Moe" + stooges; // "Moe".operator+(stooges); ERROR.

```

- Mystring operator+ (concatenation):

```

1 Mystring Mystring::operator+(const Mystring &rhs) const {
2     size_t buff_size =
3         std::strlen(str)
4         + std::strlen(rhs.str) + 1;
5     char *buff = new char[buff_size];
6     std::strcpy(buff, str);
7     std::strcat(buff, rhs.str);
8     Mystring temp {buff};
9     delete[] buff;
10    return temp;
11 }

```

2.5.3 The complete code would look like this

```

1 #include <iostream>
2 #include <vector>
3 #include <cstring>
4
5 using namespace std;
6

```



```

7 class Mystring {
8 private:
9     char *str;
10 public:
11     Mystring();
12     Mystring(const char *s);
13     Mystring(const Mystring &source);
14     ~Mystring();
15
16     // copy assignment (overloaded).
17     Mystring &operator=(const Mystring &rhs);
18     // move assignment (overloaded).
19     Mystring & operator=(Mystring &&rhs);
20     // operator minus to make lower case.
21     Mystring operator-() const;
22     // operator plus to make concatenation.
23     Mystring operator+(const Mystring &rhs) const;
24     // equality operator ==
25     bool operator==(const Mystring &rhs) const;
26
27     void display() const;
28     int get_length() const;
29     const char *get_str() const;
30 };
31 // no args constructor.
32 Mystring::Mystring() : str(nullptr) {
33     // Make an empty string
34     str = new char[1];
35     *str = '\0';
36 }
37
38 // overloaded constructor.
39 Mystring::Mystring(const char *s) : str(nullptr) {
40     if (s == nullptr) {
41         str = new char[1];
42         *str = '\0';
43     }
44     else {
45         str = new char[strlen(s) + 1];
46         strcpy(str, s);
47     }
48 }
49
50 // copy constructor
51 Mystring::Mystring(const Mystring &source)
52 : str(nullptr) {
53     str = new char[strlen(source.str)+ 1];
54     strcpy(str, source.str);
55     std::cout << "Copy constructor used" << std::endl;
56 }
57
58
59
60 // destructor.

```

```

61 Mystring::~Mystring() {
62     delete[] str;
63 }
64
65 // print the string.
66 void Mystring::display() const {
67     std::cout << str << ": " << get_length() << std::endl;
68 }
69
70 // string length.
71 int Mystring::get_length() const { return std::strlen(str); }
72
73 // string getter.
74 const char *Mystring::get_str() const { return str; }
75
76 // copy assignment operator overloading.
77 Mystring &Mystring::operator=(const Mystring &rhs) {
78     // std::cout << "Copy assignment" << std::endl;
79     if (this == &rhs) {
80         return *this;
81     }
82     delete[] this->str;
83     str = new char[strlen(rhs.str) + 1];
84     strcpy(this->str, rhs.str);
85     return *this;
86 }
87
88 // move assignment operator.
89 Mystring &Mystring::operator=(Mystring &&rhs) {
90     if (this == &rhs) { // self assignment.
91         return *this; // return current object.
92     }
93     delete[] str; // deallocate current storage.
94     str = rhs.str; // steal the pointer
95     rhs.str = nullptr; // null out the rhs object.
96     return *this;
97 }
98
99 // equality operator.
100 bool Mystring::operator==(const Mystring &rhs) const {
101     return (std::strcmp(this->str, rhs.str) == 0);
102 }
103
104 // make lower case with -
105 Mystring Mystring::operator-() const {
106     char *buff = new char[std::strlen(str) + 1];
107     std::strcpy(buff, str);
108     for (size_t i {0}; i < std::strlen(buff); i++) {
109         buff[i] = std::tolower(buff[i]);
110     }
111     Mystring temp {buff};
112     delete[] buff;
113     return temp;
114 }

```

```

115
116 // concatenate with +
117 Mystring Mystring::operator+(const Mystring &rhs) const {
118     char *buff = new char[std::strlen(str) + std::strlen(rhs.str) + 1];
119     std::strcpy(buff, str);
120     std::strcat(buff, rhs.str);
121     Mystring temp {buff};
122     delete[] buff;
123     return temp;
124 }
125
126
127 int main() {
128     cout << std::boolalpha << endl;
129     Mystring larry{"Larry"};
130     Mystring moe{"Moe"};
131
132     Mystring stooge = larry;
133     larry.display();
134     moe.display();
135
136     std::cout << (larry == moe) << std::endl;
137     std::cout << (larry == stooge) << std::endl;
138
139     larry.display();
140     Mystring larry2 = -larry;
141     larry2.display();
142
143     Mystring stooges = larry + "Moe";
144
145     Mystring two_stooges = moe + " " + "Larry";
146
147     return 0;
148 }
149
150 /* OUTPUT:
151 Copy constructor used
152 Larry: 5
153 Moe: 3
154 false
155 true
156 Larry: 5
157 larry: 5
158 */

```

2.6 Overloading operators as Global functions

- Overloading operators can be done by member functions (by methods that describe interactions with multiple instances of a class) or by global functions.
- Some operators - for example the unary operators - need to be overloaded using global functions. The reason for this is basically because you do not have two operands.
- These are not member functions, therefore we no longer have a `this` pointer referring to the object in

the left hand side. These operators don't really need access to private attributes in the class, and thus they are overloaded using non member functions. If they must do anything with private attributes it is usually just the matter of declaring them to be friends of the class.

- In the case of a unary operator, only a single parameter is specified in the parameter list.
- Example:

```
1 Mystring operator-(const Mystring &obj) {
2     char *buff = new char[std::strlen(obj.str) + 1];
3     std::strcpy(buff, obj.str);
4     for (size_t i {0}; i < std::strlen(buff); i++) {
5         buff[i] = std::tolower(buff[i]);
6     }
7     Mystring temp {buff};
8     delete[] buff;
9     return temp;
10 }
```

2.6.1 Binary operators as global functions

- +, -, ==, <, >, etc:

Return Type operatorOp(const Type &lhs, const Type &rhs);

```
1 Number operator+(const Number &lhs, const Number &rhs);
2 Number operator-(const Number &lhs, const Number &rhs);
3 Number operator==(const Number &lhs, const Number &rhs);
4 Number operator<(const Number &lhs, const Number &rhs);
5
6 Number n1 {100}, n2 {200};
7 Number n3 = n1 + n2; // operator+(n1, n2)
8 n3 = n1 - n2; // operator-(n1, n2)
9 if (n1 == n2) { ... } // operator==(n1, n2)
```

- This following example assumes that the function being overloaded is a friend of the Mystring class since we are accessing private attributes, otherwise we would need getters and setters. Overloading the `operator==`

```
1 bool operator==(const Mystring &lhs, const Mystring &rhs) {
2     if (std::strcmp(lhs.str, rhs.str) == 0) {
3         return true;
4     }
5     else {
6         return false;
7     }
8 }
```

- The following example is overloading the `operator+` for concatenation as a non-member function:

```
1 Mystring larry {"Larry"};
2 Mystring moe {"Moe"};
3 Mystring stooges {" is one of the three stooges"};
```

```

4
5 Mystring result = larry + stooges; // operator+(larry, stooges);
6
7 result = moe + " is also a stooge"; // operator+(moe, "is also a stooge")
8
9 result = "Moe" + stooge; // ok with non member functions.

```

- The order of the arguments with non member functions is irrelevant, with member functions we needed the left to be an object in order to perform the concatenation.

```

1 Mystring operator+(const Mystring &lhs, const Mystring &rhs) {
2     size_t buff_size = std::strlen(lhs.str) + std::strlen(rhs.str) + 1;
3     char *buff = new char[buff_size];
4     std::strcpy(buff, lhs.str);
5     std::strcat(buff, rhs.str);
6     Mystring temp {buff};
7     delete[] buff;
8     return temp;
9 }

```

2.6.2 Declaring friends of a class

- Friends need to be declared inside the class, it doesn't matter where you declare them, you can declare them inside the private area, public area, or the protected area.
- Example:

```

1 class MyClass {
2     friend bool func1() {};
3     friend bool func2() {};
4 };

```

2.6.3 Example

```

1 #include <iostream>
2 #include <vector>
3 #include <cstring>
4
5 using namespace std;
6
7 class Mystring {
8     friend bool operator==(const Mystring &lhs, const Mystring &rhs);
9     friend Mystring operator-(const Mystring &obj);
10    friend Mystring operator+(const Mystring &lhs, const Mystring &rhs);
11 private:
12    char *str;
13 public:
14    Mystring();
15    Mystring(const char *s);
16    Mystring(const Mystring &source);
17    ~Mystring();
18
19    // copy assignment (overloaded).

```

```

20     Mystring &operator=(const Mystring &rhs);
21     // move assignment (overloaded).
22     Mystring & operator=(Mystring &&rhs);
23
24     void display() const;
25     int get_length() const;
26     const char *get_str() const;
27 };
28 // no args constructor.
29 Mystring::Mystring() : str(nullptr) {
30     // Make an empty string
31     str = new char[1];
32     *str = '\0';
33 }
34 // overloaded constructor.
35 Mystring::Mystring(const char *s) : str(nullptr) {
36     if (s == nullptr) {
37         str = new char[1];
38         *str = '\0';
39     }
40     else {
41         str = new char[strlen(s) + 1];
42         strcpy(str, s);
43     }
44 }
45 // copy constructor
46 Mystring::Mystring(const Mystring &source)
47     : str(nullptr) {
48     str = new char[strlen(source.str)+ 1];
49     strcpy(str, source.str);
50     std::cout << "Copy constructor used" << std::endl;
51 }
52 // destructor.
53 Mystring::~Mystring() {
54     delete[] str;
55 }
56
57 // print the string.
58 void Mystring::display() const {
59     std::cout << str << ": " << get_length() << std::endl;
60 }
61
62 // string length.
63 int Mystring::get_length() const { return std::strlen(str); }
64
65 // string getter.
66 const char *Mystring::get_str() const { return str; }
67 // copy assignment operator overloading.
68 Mystring &Mystring::operator=(const Mystring &rhs) {
69     // std::cout << "Copy assignment" << std::endl;
70     if (this == &rhs) {
71         return *this;
72     }
73     delete[] this->str;

```

```

74     str = new char[strlen(rhs.str) + 1];
75     strcpy(this->str, rhs.str);
76     return *this;
77 }
78 // move assignment operator.
79 Mystring &Mystring::operator=(Mystring &&rhs) {
80     if (this == &rhs) { // self assignment.
81         return *this;    // return current object.
82     }
83     delete[] str; // deallocate current storage.
84     str = rhs.str; // steal the pointer
85     rhs.str = nullptr; // null out the rhs object.
86     return *this;
87 }
88 // Equality non-member function
89 bool operator==(const Mystring &lhs, const Mystring &rhs) {
90     return (std::strcmp(lhs.str, rhs.str) == 0);
91 }
92 // Make lowercase non-member function
93 Mystring operator-(const Mystring &obj) {
94     char *buff = new char[std::strlen(obj.str) + 1];
95     std::strcpy(buff, obj.str);
96     for (size_t i {0}; i < std::strlen(buff); i++) {
97         buff[i] = std::tolower(buff[i]);
98     }
99     Mystring temp {buff};
100    delete[] buff;
101    return temp;
102 }
103 // Concatenation non-member function
104 Mystring operator+(const Mystring &lhs, const Mystring &rhs) {
105     char *buff = new char[std::strlen(lhs.str) + std::strlen(rhs.str) + 1];
106     std::strcpy(buff, lhs.str);
107     std::strcat(buff, rhs.str);
108     Mystring temp {buff};
109     delete[] buff;
110     return temp;
111 }
112
113 int main() {
114     cout << std::boolalpha << endl;
115     Mystring larry{"Larry"};
116     larry.display();
117
118     larry = -larry;
119     larry.display();
120
121     Mystring moe{"Moe"};
122
123     Mystring stooge = larry;
124
125     std::cout << (larry == moe) << std::endl;
126     std::cout << (larry == stooge) << std::endl;
127

```

```

128     Mystring stooges = "Larry" + moe;
129     stooges.display();
130
131     Mystring two_stooges = moe + " " + "Larry";
132     two_stooges.display();
133
134     return 0;
135 }
136
137 /* OUTPUT:
138 Larry: 5
139 larry: 5
140 Copy constructor used
141 false
142 true
143 */

```

2.7 Overloading the stream insertion and extraction operators

- Overloading these operators mean we can now print and insert strings to our object very much like the `std::cout`.
- Examples:

```

1 // <<
2 Mystring larry {"Larry"};
3 cout << larry << endl; // Larry
4 Player hero {"Hero", 100, 33};
5 cout << hero << endl; // {name: hero, health: 100, xp:33}
6
7 // >>
8 Mystring larry;
9 cin >> larry;
10 Player hero;
11 cin >> hero;

```

- Stream insertion and extraction operators are usually overloaded using non member functions, this makes sense because:

- It doesn't make sense to implement as member methods:

- * The left operand must be a user-defined class of the type we are overloading.

```

1 Mystring larry;
2 larry << cout; // ???

```

- * Not the way we normally use these operators.

```

1 Player hero;
2 hero >> cin; // ???

```

- * You can overload them as member functions, however you can overload them as member functions and use the `hero << cout`; code, but it will be weird and not many will understand.

- Example `<<`:


```

1 std::ostream &operator<<(std::ostream &os, const Mystring &obj) {
2     os << obj.str; // if friend function.
3     // os << obj.get_str(); // if not a friend function.
4     return os;
5 }

```

- Return a reference to the `ostream` so we can keep inserting.
- Don't return `ostream` by value! It needs to be a reference, otherwise you are copying the entire stream.

- Example >>:

```

1 std::istream & operator>>(std::istream &is, Mystring &obj) {
2     char *buff = new char[1000];
3     is >> buff;
4     obj = Mystring(buff); // If you have a copy or move assignment.
5     delete[] buff;
6     return is;
7 }

```

- Return a reference to the `istream` so we can keep inserting.
- Update the object passed in.

Chapter 3

Inheritance